Inventory Management System for B2B SaaS

Overview

You're joining a team building "StockFlow" - a B2B inventory management platform. Small businesses use it to track products across multiple warehouses and manage supplier relationships.

Part 1: Code Review & Debugging (30 minutes)

A previous intern wrote this API endpoint for adding new products. **Something is wrong** - the code compiles but doesn't work as expected in production.

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

Your Tasks:

1. **Identify Issues**: List all problems you see with this code (technical and business logic)
2. **Explain Impact**: For each issue, explain what could go wrong in production
3. **Provide Fixes**: Write the corrected version with explanations

Additional Context (you may need to ask for more):

- Products can exist in multiple warehouses
- SKUs must be unique across the platform
- Price can be decimal values
- Some fields might be optional

Part 2: Database Design (25 minutes)

Based on the requirements below, design a database schema. **Note**: These requirements are intentionally incomplete - you should identify what's missing.

Given Requirements:

- Companies can have multiple warehouses
- Products can be stored in multiple warehouses with different quantities
- Track when inventory levels change
- Suppliers provide products to companies
- Some products might be "bundles" containing other products

Your Tasks:

1. **Design Schema**: Create tables with columns, data types, and relationships
2. **Identify Gaps**: List questions you'd ask the product team about missing requirements
3. **Explain Decisions**: Justify your design choices (indexes, constraints, etc.)

Format: Use any notation (SQL DDL, ERD, text description, etc.)

Part 3: API Implementation (35 minutes)

Implement an endpoint that returns low-stock alerts for a company.

Business Rules (discovered through previous questions):

- Low stock threshold varies by product type
- Only alert for products with recent sales activity
- Must handle multiple warehouses per company
- Include supplier information for reordering

Endpoint Specification:

GET /api/companies/{company_id}/alerts/low-stock

Expected Response Format:

```
{
  "alerts": [
    {
      "product_id": 123,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 456,
      "warehouse_name": "Main Warehouse",
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 12,
      "supplier": {
        "id": 789,
        "name": "Supplier Corp",
        "contact_email": "orders@supplier.com"
      }
    }
  ],
  "total_alerts": 1
}
```

Your Tasks:

1. **Write Implementation**: Use any language/framework (Python/Flask, Node.js/Express, etc.)
2. **Handle Edge Cases**: Consider what could go wrong
3. **Explain Approach**: Add comments explaining your logic

Hints: You'll need to make assumptions about the database schema and business logic. Document these assumptions.

Part 1: Code Review & Debugging

Issues, Impact & Fixes

1. No Input Validation

Issue:

data['name'], data['sku']

Direct access without checking if fields exist.

Impact:

- API crashes with `KeyError` if any field is missing
- Bad user experience (500 errors instead of proper message)

Fix:
Use `.get()` and validate required fields:

```
name = data.get('name')
if not name:
  return {"error": "Name is required"}, 400
```

2. SKU Uniqueness Not Checked

Issue:
No validation for duplicate SKU.

Impact:

- Duplicate products with same SKU
- Inventory confusion (SKU is a unique identifier in business)

Fix:

```
if Product.query.filter_by(sku=data['sku']).first():
  return {"error": "SKU already exists"}, 409
```

3. No Transaction Management (Multiple Commits)

Issue:

db.session.commit()

```
db.session.commit()
```

Impact:

- Product saved but inventory fails → inconsistent database
- Leads to orphan records

Fix:
Use single transaction:

```
db.session.flush()
db.session.commit()
```

OR wrap in try-except with rollback

4. Wrong Business Logic (Product tied to One Warehouse)

Issue:

```
warehouse_id=data['warehouse_id']
```

Impact:

- Violates requirement: product should exist in multiple warehouses
- Poor database design

Fix:

- Remove `warehouse_id` from Product
- Handle warehouse in Inventory table only

5. No Error Handling

Issue:
No try-except block.

Impact:

- Any DB error → server crash
- No proper error message returned

Fix:

```
try:
  ...
except Exception:
  db.session.rollback()
```

```
    return {"error": "Something went wrong"}, 500
```

6. Price Not Properly Handled

Issue:

```
price=data['price']
```

Impact:

- Float precision issues (financial data incorrect)
- Invalid values can be stored

Fix:

```
from decimal import Decimal
price = Decimal(str(data['price']))
```

7. No Validation for Quantity

Issue:

```
quantity=data['initial_quantity']
```

Impact:

- Negative inventory possible
- Missing field causes crash

Fix:

```
quantity = data.get('initial_quantity', 0)
if quantity < 0:
    return {"error": "Quantity cannot be negative"}, 400
```

8. Warehouse Not Validated

Issue:
No check if warehouse exists.

Impact:

- Inventory linked to invalid warehouse
- Data integrity issues

Fix:

```
warehouse = Warehouse.query.get(data['warehouse_id'])
```

```
if not warehouse:
  return {"error": "Invalid warehouse"}, 400
```

9. No Handling of Optional Fields

Issue:
Assumes all fields always exist.

Impact:

- API becomes rigid
- Fails when optional data is missing

Fix:

```
warehouse_id = data.get('warehouse_id')
```

10. Poor API Response Design

Issue:

```
return {"message": "Product created"}
```

Impact:

- No HTTP status code
- Not useful for frontend

Fix:

```
return {"message": "Product created", "product_id": product.id}, 201
```

Corrected code

```
@app.route('/api/products', methods=['POST'])
def create_product():
   data = request.get_json()

   if not data:
      return {"error": "Invalid request"}, 400

   name = data.get('name')
```

```
sku = data.get('sku')
price = data.get('price')

if not name or not sku or price is None:
    return {"error": "name, sku and price are required"}, 400

# Validate price
try:
    price = Decimal(str(price))
    if price < 0:
        return {"error": "Price must be positive"}, 400
except:
    return {"error": "Invalid price"}, 400

# Check SKU uniqueness
if Product.query.filter_by(sku=sku).first():
    return {"error": "SKU already exists"}, 409

try:
    # Create product
    product = Product(name=name, sku=sku, price=price)
    db.session.add(product)
    db.session.flush()

    # Optional inventory
    warehouse_id = data.get('warehouse_id')
    quantity = data.get('initial_quantity', 0)

    if warehouse_id:
        warehouse = Warehouse.query.get(warehouse_id)
        if not warehouse:
            return {"error": "Invalid warehouse"}, 400

        if quantity < 0:
            return {"error": "Quantity cannot be negative"}, 400

        inventory = Inventory(
            product_id=product.id,
            warehouse_id=warehouse_id,
            quantity=quantity
        )
        db.session.add(inventory)

    db.session.commit()

    return {
        "message": "Product created",
        "product_id": product.id
```

```
        }, 201

    except Exception:
        db.session.rollback()
        return {"error": "Something went wrong"}, 500
```

Part 2: Database Design

1. Database Schema Design (Clean & Scalable)

1. `companies`

```
CREATE TABLE companies (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. `warehouses`

```
CREATE TABLE warehouses (
  id SERIAL PRIMARY KEY,
  company_id INT NOT NULL,
  name VARCHAR(255),
  location VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (company_id) REFERENCES companies(id)
);
```

One company → multiple warehouses

3. `products`

```
CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  company_id INT NOT NULL,
  name VARCHAR(255) NOT NULL,
  sku VARCHAR(100) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  is_bundle BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  UNIQUE (sku),
  FOREIGN KEY (company_id) REFERENCES companies(id)
```

```
);
```

4. `inventory`

```
CREATE TABLE inventory (
  id SERIAL PRIMARY KEY,
  product_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  quantity INT NOT NULL DEFAULT 0,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  UNIQUE (product_id, warehouse_id),

  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);
```

Handles **same product in multiple warehouses**

5. `inventory_logs` (Tracking changes)

```
CREATE TABLE inventory_logs (
  id SERIAL PRIMARY KEY,
  product_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  change_type VARCHAR(50), -- e.g. ADD, REMOVE, ADJUST
  quantity_change INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);
```

Tracks history of inventory changes

6. `suppliers`

```
CREATE TABLE suppliers (
  id SERIAL PRIMARY KEY,
  company_id INT NOT NULL,
  name VARCHAR(255),
  contact_info TEXT,

  FOREIGN KEY (company_id) REFERENCES companies(id)
);
```

7. `supplier_products` (Many-to-Many)

```
CREATE TABLE supplier_products (
  id SERIAL PRIMARY KEY,
  supplier_id INT,
  product_id INT,
  price DECIMAL(10,2),

  FOREIGN KEY (supplier_id) REFERENCES suppliers(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

A supplier can supply multiple products

8. `product_bundles` (Bundle Support)

```
CREATE TABLE product_bundles (
  id SERIAL PRIMARY KEY,
  bundle_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,

  FOREIGN KEY (bundle_id) REFERENCES products(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

Handles **bundle → multiple products**

2. Missing Requirements (Questions to Ask)

Business Logic Questions

1. **SKU Scope**
   - Should SKU be unique globally or per company?
2. **Inventory Rules**
   - Can inventory go negative (backorders)?
3. **Warehouse Constraints**
   - Can products exist without inventory initially?
4. **Supplier Relationships**
   - Can multiple suppliers supply the same product?
   - Do we track lead time or delivery schedules?
5. **Bundle Behavior**
   - Can bundles contain other bundles?
   - Should bundle inventory be auto-calculated?
6. **Audit Requirements**
   - Do we need to track *who* made inventory changes?
7. **Multi-Tenancy**
   - Can different companies see each other's data?

8. **Pricing**
    - Is price fixed or varies by supplier/warehouse?

3. Design Decisions (Justification)

1. Separate `inventory` Table

Why?

- Supports multiple warehouses
- Keeps product table clean

2. `UNIQUE(product_id, warehouse_id)`

Prevents duplicate inventory rows

3. `inventory_logs` Table

Why?

- Audit trail
- Helps debugging stock issues
- Required for real-world systems

4. Many-to-Many (`supplier_products`)

Why?

- Real-world: multiple suppliers per product

5. Bundle Table (Self-Referencing)

Why?

- Flexible design for composite products
- Supports scalability

6. Indexing (Important!)

Add indexes for performance:

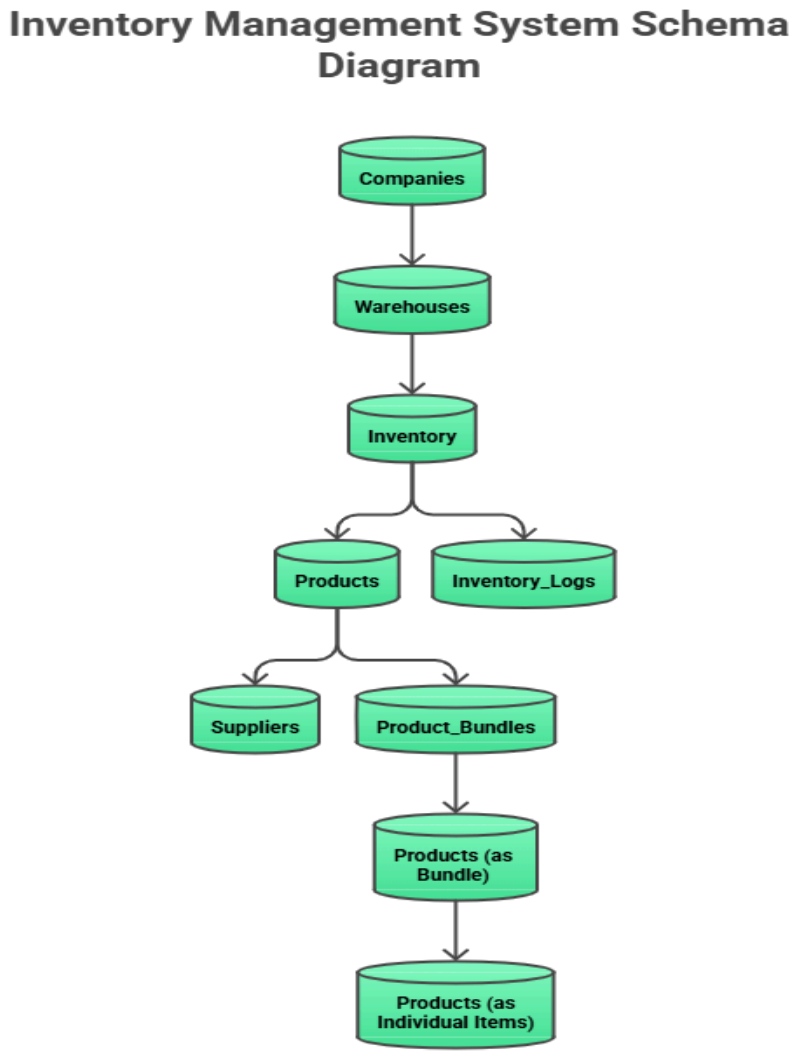
```
CREATE INDEX idx_inventory_product ON inventory(product_id);

CREATE INDEX idx_inventory_warehouse ON inventory(warehouse_id);

CREATE INDEX idx_products_sku ON products(sku);
```

7. Data Integrity via Constraints

- Foreign keys → consistency
- Unique constraints → prevent duplicates



Part 3: API Implementation

```
from flask import jsonify
from datetime import datetime, timedelta

@app.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])
def get_low_stock_alerts(company_id):
    alerts = []

    try:
        # --- Step 1: Get all warehouses of the company ---
        warehouses = Warehouse.query.filter_by(company_id=company_id).all()

        if not warehouses:
            return jsonify({"alerts": [], "total_alerts": 0}), 200

        warehouse_ids = [w.id for w in warehouses]

        # --- Step 2: Get inventory for those warehouses ---
        inventories = Inventory.query.filter(
            Inventory.warehouse_id.in_(warehouse_ids)
        ).all()

        for inv in inventories:
            product = Product.query.get(inv.product_id)
            warehouse = Warehouse.query.get(inv.warehouse_id)

            # --- Step 3: Get product threshold ---
            threshold = product.low_stock_threshold or 10  # default = 10

            # --- Step 4: Check recent sales (last 30 days assumed) ---
            recent_sales = Sales.query.filter(
                Sales.product_id == product.id,
                Sales.created_at >= datetime.utcnow() - timedelta(days=30)
            ).all()

            if not recent_sales:
                continue  # skip if no recent sales

            # --- Step 5: Check low stock condition ---
            if inv.quantity >= threshold:
                continue

            # --- Step 6: Calculate average daily sales ---
            total_sold = sum(s.quantity for s in recent_sales)
            avg_daily_sales = total_sold / 30 if total_sold > 0 else 0
```

```
        if avg_daily_sales == 0:
            days_until_stockout = None
        else:
            days_until_stockout = int(inv.quantity / avg_daily_sales)

        # --- Step 7: Get supplier info (first supplier assumed) ---
        supplier_link = SupplierProduct.query.filter_by(
            product_id=product.id
        ).first()

        supplier_data = None
        if supplier_link:
            supplier = Supplier.query.get(supplier_link.supplier_id)
            supplier_data = {
                "id": supplier.id,
                "name": supplier.name,
                "contact_email": supplier.contact_email
            }

        # --- Step 8: Build alert object ---
        alert = {
            "product_id": product.id,
            "product_name": product.name,
            "sku": product.sku,
            "warehouse_id": warehouse.id,
            "warehouse_name": warehouse.name,
            "current_stock": inv.quantity,
            "threshold": threshold,
            "days_until_stockout": days_until_stockout,
            "supplier": supplier_data
        }

        alerts.append(alert)

    return jsonify({
        "alerts": alerts,
        "total_alerts": len(alerts)
    }), 200

except Exception as e:
    return jsonify({"error": "Something went wrong"}), 500
```

2. Assumptions

- Each **product has a `low_stock_threshold` field**
- Sales data is stored in a `Sales` table
- "Recent sales" = **last 30 days**
- One product can have multiple suppliers → we pick the **primary/first supplier**
- `days_until_stockout` = current stock ÷ avg daily sales
- If no sales → no alert (as per requirement)

3. Edge Cases Handled

1. No Warehouses

if not warehouses:

Returns empty response

2. No Recent Sales

```
if not recent_sales:
  continue
```

Avoid unnecessary alerts

3. Zero Sales (Division Issue)

if avg_daily_sales == 0:

Prevent division by zero

4. Missing Supplier

supplier_data = None

Avoid crashes if supplier not found

5. Default Threshold

threshold = product.low_stock_threshold or 10
Handles missing values